Lab 3 — Component Modelling & Architectural Pattern Selection

Samarth Rajesh Rao | PES1UG24AM244

Municipal Infrastructure Damage Reporting App · Smart Cities, Transport & Logistics · Actors: Citizen Reporter, Municipal Engineer

---

Architecture Selection: We chose a Microservices architecture for the Municipal Infrastructure Damage Reporting App.

Requirements: Functional — citizens submit geo-tagged, photo-attached damage reports and track status; engineers verify reports, assign work orders and update resolution. Non-functional — high availability for a city-wide public service, spiky scalability, secure handling of citizen data, and tamper-evident audit integrity.

Style analysis: Layered is simple with clear separation but deploys and scales as one unit, so a report-ingestion spike degrades the whole app. Client-Server gives easy consistency but the central server is a scalability bottleneck and a single point of failure for a city-wide service. Microservices offers independently deployable, independently scalable services with fault isolation — selected.

Two reasons for choosing microservices architecture:

- **Independent scaling —** citizen report volume is highly spiky (storms, floods, festivals). The Reporter Portal and Assessment Engine scale horizontally on their own, without over-provisioning the engineer-facing services.
- **Fault isolation —** if the SLA & Notification service fails, citizens can still file reports and engineers can still triage; no single failure takes down the whole civic service.

Security advantage: All services sit behind an API gateway that centralises authentication, rate-limiting and input validation, and the public Citizen Portal is network-isolated from the internal Work-Order and Database services — so compromising the public endpoint does not expose municipal work-order data or the cryptographic audit trail.

Performance benefit: Report ingestion is asynchronous — the citizen gets an instant “report received” acknowledgement while classification, de-duplication and routing run in the background, keeping latency low under heavy load, with each service tuned and scaled to its own workload.

Components (9): Citizen Reporter Portal · Municipal Engineer Portal · API Gateway · Damage Assessment & Triage Service · Work Order Management Service · SLA & Notification Service — each with its own private database (Assessment DB, Work Order DB, Notification DB) — plus a shared, event-sourced Audit & Reporting Store.

Interfaces (≥4): submitReport() (Citizen → API Gateway, sync) · assignWorkOrder()/reviewReport() (API Gateway → Engineer Portal, sync) · routeReport()/Work Order API/Status Query API (API Gateway → each service, sync) · Data Access (each service → its own database, sync) · publish event / async (each service → Audit & Reporting Store, decoupled).

